Enum cases behave like subclasses of an abstract parent, where the parent is the enum. Thus, all cases of `WizardSpells` have "inherited" the `cast()` method. Just like abstract classes, an enum cannot be instantiated…

```
$spells = new WizardSpells();
```

```
Output:
Fatal error: Uncaught Error: Cannot instantiate enum WizardSpells
```

… and of course we can't instantiate an enum case because it is already an object. It is possible to define an enum method that can be called on the enum itself provided we declare the method to be `static`. Let's add the following to `WizardSpells`:

```
    public static function panic(): void
    {
        print "You panic and fall over." . PHP_EOL;
    }
```

Now we can call `panic()` from either a case or the enum:

```
WizardSpells::panic();
print "You recover and cast a noisy spell... "
    . WizardSpells::Thunderwave->value . PHP_EOL;
WizardSpells::Thunderwave->panic();
```

```
Output:
You panic and fall over.
You recover and cast a noisy spell... KABOOM!
You panic and fall over.
```

There's one more trick up the sleeve of enums: it is possible to define constants. These function in the same way as class constants.

```
enum BardInstrument
{
    case Bagpipes;
    case Harp;
    case Zither;

    public const DEFAULT = self::Harp;
}
```